

0.105. Une lettre qui tire — le tir casse le X

On reprend le 0.101 (le mur de X, un seul tir) : le tir qui arrive sur un X le casse. Le X disparaît, le tir aussi ; le tir suivant passe par le trou, et le A aussi.

1. Ce qu'il faut comprendre

On revient au mur du 0.101, pour le casser. Au 0.101, le X arrêta le tir, et restait. Ici, le tir qui arrive sur un X le **casse** : le X disparaît, le tir aussi. Le mur a un **trou** : le tir suivant passe, et le A peut y marcher.

Ce qui vient d'avant : tout le 0.101. Le A bouge et regarde (le 0.100), le tir part dans son sens, et avant chaque pas il regarde sa case d'arrivée avec `lire()`.

Ce qui est nouveau ici : un cas de plus, « c'est un X ». Le pas du tir avait trois cas : hors de l'écran, pas vide, vide. On en ajoute un, **avant** « pas vide » : `lire(nx, ny) == ALPHABET[23]`, l'arrivée est un X. Alors `effacer(nx, ny, 1)` efface le X, et `vole = 0` : le tir s'est usé dessus, il disparaît.

Pourquoi avant « pas vide » ? Un X n'est pas vide : si « pas vide » venait d'abord, le tir s'y écraserait comme au 0.101, et le cas du X ne serait jamais regardé. Dans une suite de `else if`, **le premier cas vrai gagne**.

Déroulons : A en (2, 8), regard à droite. Bouton A : le tir va de (3, 8) à (11, 8). Pas suivant : l'arrivée (12, 8) est un X. On efface (12, 8), le tir disparaît. Bouton A encore : le tir passe (12, 8), qui est vide, et file jusqu'au bord. DROITE : le A passe aussi par le trou.

Essaie : fais que le tir traverse le X sans disparaître : dans le cas du X, remplace `vole = 0;` par ce que fait le cas « vide » (`tx = nx; ty = ny;` et le dessin). Un tir perçant !

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      85 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
main()         485 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
```

AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer(), texte()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	49 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	48 o	← CopierTuiles	– 3 lettres sur 43
","	1 o	← texte()	– un texte gravé
","	1 o	← texte()	– un texte gravé

Lettres gravées : AX.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

effacer()	× 3	45 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
lire()	× 4	44 o sur place, dans main()	+ AdresseCase (10 o)
poser()	× 3	38 o sur place, dans main()	+ AdresseCase (10 o)
texte()	× 2	36 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
bouton()	× 5	25 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le tir casse le X ----
// Le 0.101 (le mur de X), mais le mur ne tient plus : le tir qui arrive
// sur un X le CASSE. Le X disparaît, le tir aussi, et le mur a un trou.
// ALPHABET[23] : le X.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l'écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;          // le sens où regarde le A (le 0.100)
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;          // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UNE VARIABLE — TX

```
uint8_t tx = 0;           // le tir : sa place...
```

9. UNE VARIABLE — TY

```
uint8_t ty = 0;
```

10. UNE VARIABLE — TDX

```
uint8_t tdx = 0;         // ...et son sens
```

11. UNE VARIABLE — TDY

```
uint8_t tdy = 0;
```

12. UNE VARIABLE — VOLE

```
uint8_t vole = 0;        // 1 = le tir est en vol ; 0 = pas de tir
```

13. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- NOUVEAU : un mur de X, debout, en colonne 12, des lignes 4 à 12 ----
    for (uint8_t l = 4; l < 13; l++) {    // l vaut 4, 5, 6... jusqu'à 12 : 9 X
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde (le 0.100) ----
        // Il ne va que sur une case vide : le mur l'arrête déjà.
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- A lance le tir ----
        if (bouton(A) && vole == 0) {
            nx = ax + dx;
            ny = ay + dy;
            // NOUVEAU : la case de départ doit aussi être VIDE.
            // Collé au mur, le A ne tire pas : le tir n'a pas de place.
            if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                vole = 1;
                tx = nx;
                ty = ny;
                tdx = dx;
                tdy = dy;
                texte(tx, ty, ".");
            }
        }

        // ---- le tir avance ; il regarde sa case d'arrivée AVANT d'y aller ----
        if (chaque(100)) {
            if (vole == 1) {
                nx = tx + tdx;
                ny = ty + tdy;
                effacer(tx, ty, 1);
            }
        }
    }
}
```

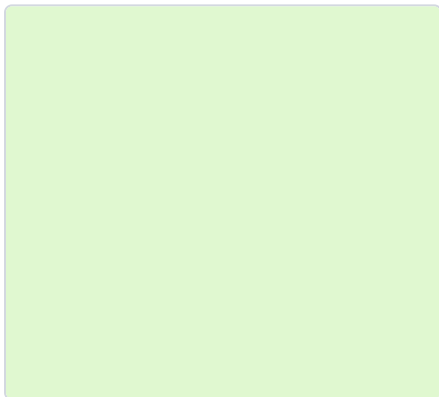
```

if (nx >= 20 || ny >= 17) {
    vole = 0; // 1. hors de l'écran : il disparaît
} else if (lire(nx, ny) == ALPHABET[23]) {
    // 2. NOUVEAU : l'arrivée est un X : le tir le CASSE.
    effacer(nx, ny, 1); // le X disparaît (sa case devient vide)...
    vole = 0; // ...et le tir aussi : il s'est usé dessus.
} else if (lire(nx, ny) != 0) {
    vole = 0; // 3. autre chose, pas vide : il s'écrase.
} else {
    tx = nx; // 4. l'arrivée est vide : il y va
    ty = ny;
    texte(tx, ty, ".");
}
// L'ordre compte : d'abord « hors de l'écran ? », car lire(20, 8)
// n'a pas de sens : la case n'existe pas.
}
}
}
}

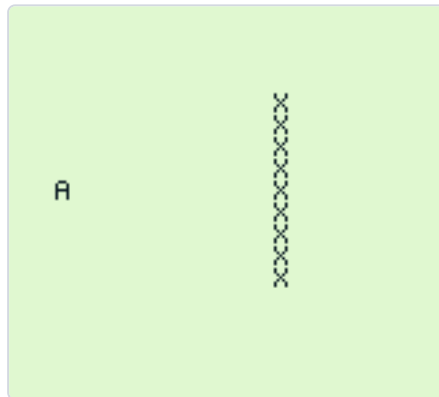
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



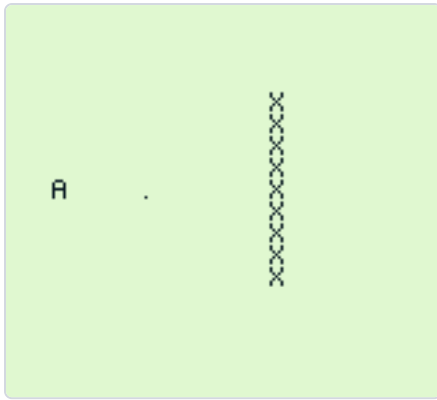
la toute première image — la console vient de démarrer — image 1



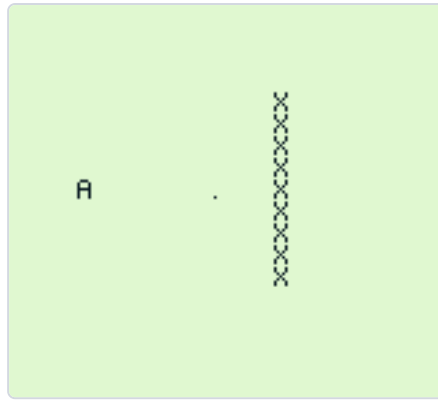
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

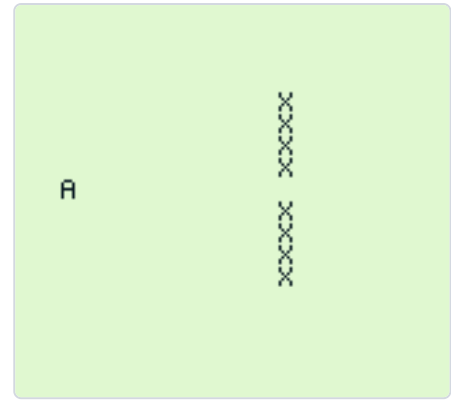
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



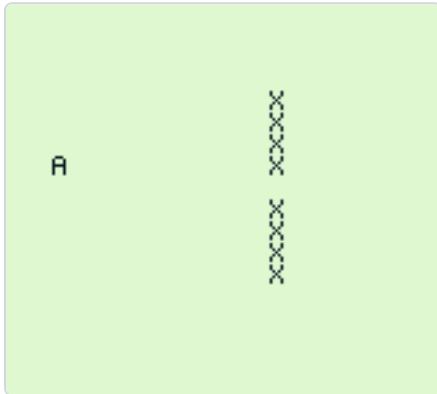
A — l'écran a changé



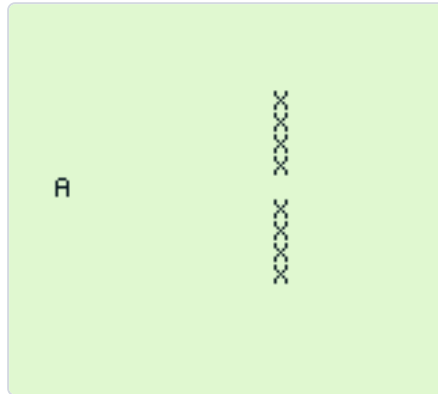
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le tir casse le X de la ligne 8 et disparaît ; le tir suivant passe par le trou jusqu'au bord, et le A peut y marcher. Les autres X restent.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, un X en (12, 8)
- ✓ le premier tir le casse : (12, 8) est vide, les X au-dessus et au-dessous restent
- ✓ le tir suivant passe par le trou, jusqu'en colonne 19 (le plus loin : colonne 19)
- ✓ le A passe par le trou, lui aussi

Ensuite : **0.105.1. Une lettre qui tire — la rafale perce le mur** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++